

DEV2 – JAVL – Laboratoire Java

TD 05 – JUnit – Les tests unitaires

Résumé

Dans la pratique, il est impossible d'écrire directement un programme sans faute. Une étape importante de tout développement d'application est de détecter l'ensemble des erreurs qu'il contient pour pouvoir les corriger.

La notion d'erreur englobe différents concepts. Ici, nous nous attacherons à vérifier que chaque **méthode** fournit bien les résultats attendus; ce qu'on appelle des **tests unitaires**.

Table des matières

1	Comment tester un programme ?	2
1.1	Les types d'erreurs	2
1.2	Tester quoi et quand ?	2
1.3	Une approche naïve	3
1.3.1	Appels successifs à la méthode à tester	3
1.3.2	Incorporation de la bonne réponse	3
1.3.3	Filtrage du résultat	3
2	JUnit	4
2.1	Découverte	4
2.2	Cas particuliers	7
2.2.1	Une méthode qui retourne un booléen	7
2.2.2	Une méthode qui lance une exception	7
2.2.3	Une méthode qui modifie l'état	8
2.2.4	Une méthode qui retourne un nombre flottant	8
2.2.5	Les entrées-sorties	8
2.2.6	L'aléatoire	8
2.3	Exercices	9
3	Plan de tests	9
4	Tests et refactoring	10
5	Développement dirigé par les tests	12

1 Comment tester un programme ?

Avant d'aborder JUNIT, posons-nous la question des types d'erreurs qu'un programme peut comporter, de ce que veut dire *tester un programme* et pourquoi une automatisation est nécessaire.

1.1 Les types d'erreurs

Un programme peut mener à trois sortes d'erreurs¹ :

1. il peut ne pas compiler ;
2. il peut se planter à l'exécution (par exemple si on demande une division par 0) ;
3. il peut fournir un résultat incorrect.

Le 1^{er} cas est facile à détecter puisqu'il suffit au développeur de demander une compilation² pour obtenir la liste complète des erreurs de ce type.

Les deux derniers cas sont plus délicats. Comment savoir qu'on a bien testé toutes les parties de code ? Qu'on a bien testé tous les cas particuliers ? Cela demande une réflexion, une **planification** des cas à tester.

1.2 Tester quoi et quand ?

Commençons par quelques réflexions sur les tests.

1. Nous allons **tester chaque méthode séparément**. L'idée est que si toutes les méthodes sont correctes, le programme le sera aussi³.
2. Pour **chaque méthode**, il faut imaginer tous les cas qui pourraient poser problème. Concrètement, il faut trouver les paramètres à donner à la fonction pour que **tous les cas particuliers** soient testés. Par exemple, si une méthode doit réagir différemment si son paramètre est négatif ou pas, il faudra au minimum la tester avec un nombre négatif et un nombre non négatif.
3. Dans une situation réelle, une application est composée de milliers de méthodes. Si chacune nécessite quelques tests, le **nombre total de tests est colossal**.
4. Lorsqu'on apporte une **modification** dans le code (pour le corriger, le rendre plus lisible ou parce qu'on ajoute une fonctionnalité) il est possible qu'on crée une erreur là où il n'y en avait pas. Pour être sûr, il faut dès lors **tout retester**.

On comprend bien qu'il faut mettre en place une procédure de tests la plus **rapide** et **facile** possible si on veut pouvoir l'exécuter régulièrement. Pour toutes ces raisons, tester *à la main* chaque cas planifié n'est pas envisageable ; une seule solution : l'**automatisation**.

Automatisation des tests Automatiser les tests revient à écrire du **code de test** dont le but est de tester le code de l'application.

1. À coté de ça, il peut avoir des défauts sans être faux à proprement parler. Cela peut impacter le développeur (s'il est peu lisible) ou l'utilisateur (s'il est trop lent, trop gourmand en mémoire ou peu convivial)

2. Explicitement ou implicitement lorsqu'on l'exécute.

3. Ce qui n'est que partiellement vrai. D'autres tests, comme des **tests d'intégration** pourront suivre pour vérifier que les différentes parties du programme communiquent correctement entre-elles.

1.3 Une approche naïve

Imaginons pour l'instant que nous ne disposons pas de JUnit – que nous ne connaissons pas encore de toute façon – et imaginons à quoi pourrait ressembler un code de test.

Pour être concret, imaginons que :

1. On a ajouté à la classe `Dog` une méthode `sound()` qui retourne le son fait par le chien (plutôt que de l'afficher à l'écran).
2. On se calque sur la méthode `bark` pour dire que le cri dépend de son poids.
 - ▷ Yip yip s'il pèse moins de 10 kilos;
 - ▷ Wouf s'il pèse entre 10 et 30 kilos (inclus);
 - ▷ WOUUUF s'il pèse plus de 30 kilos.
3. On voudrait la tester avec les poids : 9, 10, 11, 29, 30 et 31.

1.3.1 Appels successifs à la méthode à tester

Une première idée est d'appeler la méthode pour chaque cas et d'afficher la réponse.

```
Dog dog1 = new Dog("chien1", 9);
System.out.println(dog1.sound());
Dog dog2 = new Dog("chien2", 10);
System.out.println(dog2.sound());
// [...]
```

Cette approche nécessite de lire la sortie et de vérifier, au cas par cas, que chaque réponse est correcte. C'est fastidieux, même pour quelques dizaines de tests.

1.3.2 Incorporation de la bonne réponse

Une amélioration consiste à inscrire la bonne réponse dans le code et à demander au programme de vérifier qu'elle correspond à ce que la méthode retourne.

```
Dog dog1 = new Dog("chien1", 9);
System.out.println("Yip yip".equals(dog1.sound()));
Dog dog2 = new Dog("chien2", 10);
System.out.println("Wouf".equals(dog2.sound()));
// [...]
```

C'est déjà mieux mais cela nécessite de parcourir une longue suite de `true` à la recherche des quelques `false` qui s'y sont glissées. Ça reste fastidieux et source d'erreurs.

1.3.3 Filtrage du résultat

Finalement, ce qui nous intéresse, ce sont les cas d'erreurs ; on peut n'afficher que ceux-là.

```
Dog dog1 = new Dog("chien1", 9);
if (!"Yip yip".equals(dog1.sound())) {
    // Afficher des détails sur l'erreur : cas testé, valeur attendue, valeur réellement reçue...
}
Dog dog2 = new Dog("chien2", 10);
if (!"Wouf".equals(dog2.sound())) {
    // Afficher des détails sur l'erreur : cas testé, valeur attendue, valeur réellement reçue...
}
// [...]
```

C'est déjà beaucoup mieux ! Malgré tout, ça nécessite de récrire constamment des codes similaires (comparaison et affichage d'un rapport d'erreurs). On pourrait écrire une fonction qui fait le boulot mais JUnit va le faire mieux que nous. On pourra se concentrer sur ce qu'il faut tester et lui s'occupera de tous les aspects pratiques.

2 JUnit

JUNIT est le *framework* le plus utilisé pour les tests unitaires en JAVA.

Framework

Plus qu'une simple bibliothèque, un framework définit une boîte à outils qui facilite le développement d'une application tout en imposant un cadre à son architecture.

JUNIT ne fait pas partie de la bibliothèque standard de JAVA. Toutefois, il est parfaitement connu par INTELLIJ qui l'installera, d'un simple clic, lors de la première utilisation.

La dernière version en date est JUNIT 5. C'est celle que nous utilisons à l'école et que nous vous conseillons également de choisir chez vous.

2.1 Découverte

Voyons les bases de l'utilisation de JUNIT pour tester les programmes JAVA.

Préalables

Avant de commencer, synchronisez votre dépôt `git` si vous en utilisez un et créez un nouveau projet (via IntelliJ) pour cette séance.

Tutoriel 1

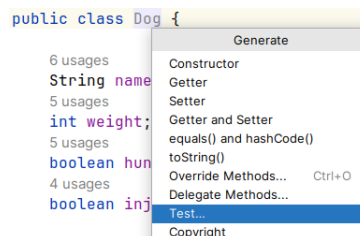
Découverte

Dans ce tutoriel, nous allons écrire des tests unitaires pour la méthode `sound()` mentionnée précédemment.

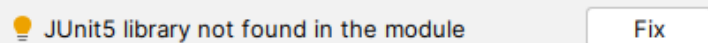
- ✍ Assurez-vous d'avoir dans votre projet, une classe `Dog` fonctionnelle.
- ✍ Ajoutez dans la classe `Dog` la méthode suivante :

```
1 String sound() {  
2     if (this.weight < 10) {  
3         return "Yip yip";  
4     } else if (this.weight < 30) {  
5         return "Wouf";  
6     } else {  
7         return "WOUUUF";  
8     }  
9 }
```

- ✍ Cliquez-droit sur le nom de la classe et choisissez **Generate.../Test...**

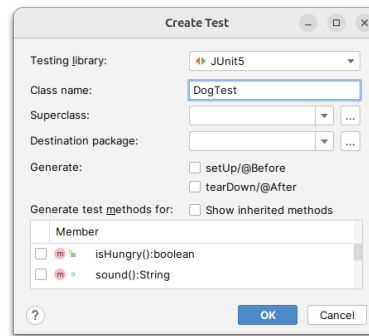


- ✍ Dans la boîte de dialogue qui s'ouvre, si c'est la 1^{re} fois, il affichera un message indiquant que JUNIT5 n'est pas encore installé.



Il suffit de cliquer sur le bouton **Fix** pour que INTELLIJ se charge de l'installation.

- ✎ Pour le reste, toutes les valeurs par défaut devraient être bonnes. Vérifiez qu'il a bien choisi JUNIT5 et cliquez sur OK.



INTELLIJ a généré pour nous une classe de tests et l'a placée dans un dossier dédié.



Par convention, les tests de la classe `Bro1` sont placés dans la classe `Bro1Test`. Vous pouvez aussi constater que quelques lignes ont été ajoutées dans le `pom.xml`.

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Ce fichier décrit le projet ; il y a ajouté la dépendance à JUNIT. Vous auriez aussi pu y placer ces lignes vous-mêmes.

- ✎ Dans la classe de tests, ajoutez un test

```
1  @Test
2  void sound_weight_9_is_small() {
3      Dog dog = new Dog("chien", 9);
4      assertEquals("Yip yip", dog.sound());
5  }
```

- ✎ Si `assertEquals` est en rouge, placez-vous sur ce mot, appuyez sur **Alt-Enter** et acceptez la 1^{ère} proposition qui va ajouter l'*import* qui va bien.

```
1  import static org.junit.jupiter.api.Assertions.*;
```

Anatomie d'un test

Que peut-on remarquer sur une méthode de test ?

- ▷ Chaque cas de test est implémenté par une méthode dédiée.
- ▷ Cette méthode de test est *annotée* par `@Test` afin d'être reconnue par JUnit.
- ▷ Elle ne prend **aucun** argument et retourne toujours `void`.
- ▷ Son nom est quelconque mais il est bon d'être le plus possible explicite car il sera utilisé dans le rapport en cas d'erreur. Nous vous suggérons d'utiliser le nom de la méthode testée, suivi d'une explication du cas testé. N'ayez pas peur des très longs noms, vous ne devrez jamais le retaper.
- ▷ Il contient une ligne qui précise le test à faire.

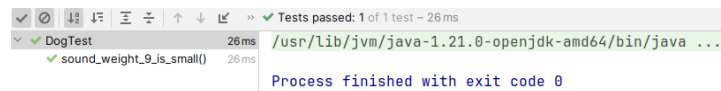
```
1 assertEquals( "Yip yip", dog.sound());
```

Cette ligne affirme (et donc demande de vérifier) que la réponse attendue ("Yip yip") est bien égale à ce que renvoie l'appel `dog.sound()`.

Il est important de mettre **d'abord la réponse attendue**, suivie de l'appel, pour que le rapport soit compréhensible en cas d'erreur.

✍ Il n'y a plus qu'à lancer le test. Pour ça, exécutez normalement la classe de test.

Vous devriez voir un rapport qui indique que tous les tests (1 sur 1) sont passés.



Tutoriel 2

Détection d'une erreur

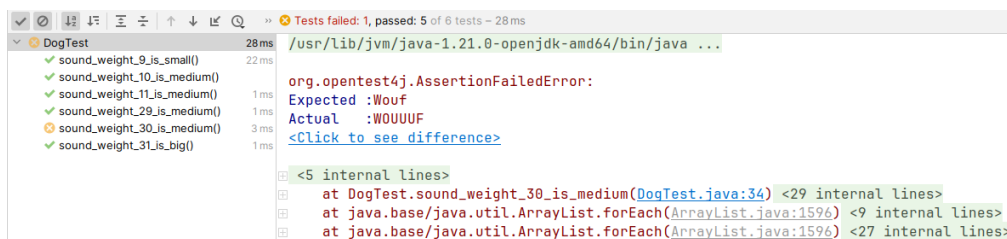
Nous allons compléter la classe de tests et voir comment sont signalées les erreurs.

✍ Ajoutez tous les autres tests prévus :

```
1  @Test
2  void sound_weight_10_is_medium() {
3      Dog dog = new Dog("chien", 10);
4      assertEquals( "Wouf", dog.sound());
5  }
6
7  @Test
8  void sound_weight_11_is_medium() {
9      Dog dog = new Dog("chien", 11);
10     assertEquals( "Wouf", dog.sound());
11 }
12
13 @Test
14 void sound_weight_29_is_medium() {
15     Dog dog = new Dog("chien", 29);
16     assertEquals( "Wouf", dog.sound());
17 }
18
19 @Test
20 void sound_weight_30_is_medium() {
21     Dog dog = new Dog("chien", 30);
22     assertEquals( "Wouf", dog.sound());
23 }
24
25 @Test
26 void sound_weight_31_is_big() {
27     Dog dog = new Dog("chien", 31);
28     assertEquals( "WOUUUF", dog.sound());
29 }
```

✎ Relancez les tests.

Cette fois-ci le rapport indique que 1 test sur les 6 a échoué.



On peut voir à gauche la liste des tests avec une icône indiquant s'il a passé ou a échoué. Et, à droite, on peut voir plus de détails sur le test qui a échoué. Notamment quelle valeur était attendue ("Wouf") et quelle valeur l'appel de fonction a réellement retourné ("WOUUUF").

Erreur dans la classe de test

Attention! Quand un test échoue, c'est peut-être dû à une erreur dans le programme mais c'est peut-être le code de test qui a été mal écrit ^a. Toujours commencer par vérifier le code de test.

^a. Non! On ne va pas écrire du code pour tester le code qui teste notre code ;) En général, un code de test est suffisamment simple que pour détecter rapidement l'erreur s'il y en a une.

Malheureusement, ici, c'est bien le code qui est erroné :(Nous ne dirons pas qui l'a écrit ;)

Exercice 1

Corriger la méthode sound()

Vous l'aurez deviné, on vous demande de corriger la méthode et de vérifier que tous les tests passent.

2.2 Cas particuliers

L'essentiel de l'utilisation de JUnit a été dit. Il reste juste quelques concepts qui vont vous permettre de faire face à toutes les situations que vous allez rencontrer en DEV2.

2.2.1 Une méthode qui retourne un booléen

Si une méthode retourne un booléen (comme la méthode `isHungry()`), on pourrait écrire

```
assertEquals( true, dog.isHungry() )
assertEquals( false, dog.isHungry() )
```

mais il existe deux méthodes dédiées pour ces situations ; préférez-les !

```
assertTrue( dog.isHungry(...) )
assertFalse( dog.isHungry(...) )
```

2.2.2 Une méthode qui lance une exception

Une méthode peut lancer une exception si elle ne parvient pas à faire ce qui lui est demandé ; typiquement parce que les paramètres ne sont pas valides. Cela fait partie des tests de vérifier qu'une méthode lance bien une exception (et la bonne) quand elle doit

le faire. Mais comment puisqu'il n'y a **aucune** valeur de retour à tester ? Une méthode qui teste une exception s'écrit ainsi ⁴ :

```
assertThrows(IllegalArgumentException.class, () -> new Dog("chien", -1));
```

On teste que le constructeur d'un `Dog` refuse bien un poids négatif.

2.2.3 Une méthode qui modifie l'état

Certaines méthodes ne retournent rien (`void`) mais modifient l'état de l'objet. Dans ce cas, on ne teste pas la valeur de retour (inexistante) mais l'état de l'objet après l'appel. Idem pour le constructeur. Par exemple, les lignes suivantes vérifient que le constructeur assigne bien le nom.

```
Dog dog = new Dog("chien", 9);
assertEquals("chien", dog.getName());
```

2.2.4 Une méthode qui retourne un nombre flottant

À cause de la précision finie des processeurs, il est délicat, lorsqu'on compare deux nombres flottants (non entiers) d'utiliser une égalité stricte. JUnit a prévu de pouvoir préciser l'écart admis entre la réponse attendue et la réponse retournée.

```
assertEquals(-1, Math.cos(Math.PI), 0.000001);
```

Ici, on vérifie la méthode `cos` de la classe `Math` fournie par la bibliothèque standard. On teste que le $\cos(\pi)$ est bien -1 en admettant un écart à partir de la 6^e décimale.

2.2.5 Les entrées-sorties

Si une méthode est censée afficher du texte à l'écran, il faudrait vérifier que ce texte s'affiche vraiment. C'est possible mais plus compliqué.

De même, si une méthode demande des valeurs à l'utilisateur (lectures au clavier), il est plus difficile de la tester.

En DEV2 on ne vous demandera de tester ni des lectures ni des affichages.

2.2.6 L'aléatoire

Comment tester une méthode qui produit un résultat aléatoire ? Pensez à la méthode `run()` développée dans le TD3 où le chien se blesse une fois sur cinq. Pour notre test, impossible de savoir si le chien doit être blessé ou pas après avoir couru.

En fait, les programmes utilisent du *pseudo-aléatoire*. Il s'agit de méthodes qui donnent l'*illusion* de l'aléatoire mais génèrent des nombres suivant un algorithme tout-à-fait déterministe (mais non prédictible). On peut contrôler ce pseudo-aléatoire en fixant la *graine* (*seed*) qui est la valeur qui sert d'amorce pour générer les nombres. Nous allons éviter ce genre de tests en DEV2.

4. Vous allez peut-être tomber sur une autre syntaxe lors de vos recherches. Attention ! Il s'agit probablement d'une vieille syntaxe qui était la seule valable avec les anciennes versions de JUnit mais qui n'est plus recommandée.

2.3 Exercices

Voici quelques tests à écrire pour la classe Dog.

Exercice 2 Un nouveau chien n'est pas blessé

Écrivez un test `new_dog_not_injured` qui vérifie qu'un chien qui vient d'être créé n'est pas blessé.

Exercice 3 Chien qui court a faim

Écrivez un test `run_after_run_is_hungry` qui vérifie qu'un chien a faim après avoir couru.

Exercice 4 Chien qui mange n'a plus faim

Écrivez un test `eat_not_hungry_anymore` qui vérifie qu'un chien n'a plus faim après avoir mangé.

Exercice 5 Chien qui a faim ne court pas

Écrivez un test `run_when_hungry_does_not_run_ISE` qui vérifie qu'un chien qui a faim refuse de courir (une `IllegalStateException` est lancée).

3 Plan de tests

Jusqu'à présent, nous avons ajouté quelques tests par-ci par-là sans y avoir vraiment réfléchi. Pour être sûr de bien tout tester et de ne pas passer à côté d'erreur, il faut **palnifier** nos tests.

Il serait tentant de limiter les tests. Moins il y en a, moins on trouvera d'erreurs dans notre code, ce qui n'est jamais agréable pour notre égo. Il faut pourtant se forcer : **le *moi* testeur doit tenter de mettre en défaut le *moi* codeur.**

Couverture de code

La *couverture de code* est le pourcentage des lignes de code qui sont effectivement testées, c'est-à-dire qui sont "visitées" par Java lors de l'exécution des tests fournis. Il faut viser une couverture de 100% mais ça ne suffit pas à garantir qu'on ne rate rien.

Lorsqu'on imagine un plan de tests il faut avoir à l'esprit les erreurs les plus communes qui sont commises quand on programme et imaginer les tests qui vont nous en prémunir en les détectant. Parmi les erreurs courantes, citons :

- ▷ Se tromper dans une comparaison en écrivant `<` au lieu de `>` (et vice-versa). ou encore `<=` au lieu de `<` (et vice-versa).
- ▷ Se tromper dans un long test en mettant un *ET* à la place d'un *OU* (et vice-versa).
- ▷ Se tromper dans le parcours d'un tableau ou d'une partie de ce tableau, en commençant trop loin ou en s'arrêtant trop tôt.
- ▷ En oubliant un `this` dans un constructeur ou un setteur ce qui a pour effet de ne pas assigner un attribut (un bon IDE nous signale ce genre d'erreur).

C'est parce que les erreurs citées plus haut sont fréquentes, qu'il est important de **tester les cas limites**. Par exemple, pour la méthode `sound()`, les valeurs de tests 5, 15 et 25 n'auraient pas permis de détecter la subtile erreur dans une comparaison.

Quel pourrait être un plan de tests complet pour la méthode `run`⁵ ?

test n°	entrées	résultat attendu	note
1	chien affamé	IllegalStateException	un chien affamé refuse de courir
2	chien blessé	IllegalStateException	un chien blessé ne peut pas courir
3	chien non affamé/non blessé	chien a faim	un chien qui court a faim

Pour le constructeur ce pourrait être :

test n°	entrées	résultat attendu	note
1	paramètres corrects	le nom est mis	initialise bien le nom du chien
2	paramètres corrects	le poids est mis	initialise bien le poids du chien
3	paramètres corrects	non affamé	un nouveau chien n'a pas faim
4	paramètres corrects	non blessé	un nouveau chien n'est pas blessé
5	poids = 0	IllegalArgumentException	un poids nul est refusé
6	poids = -1	IllegalArgumentException	un poids négatif est refusé

Un piège de plus.

À votre avis, est-il pertinent d'écrire un test pour vérifier que le poids passé au constructeur de chien est bien un entier ?

Exercice 6

Board::setLetters

Lors du TD4, vous avez écrit une méthode `setLetters()` pour la classe `Board`. Il s'agit d'une méthode plutôt difficile et il reste peut-être encore des erreurs. Écrivez un plan de tests **complet** pour cette méthode et écrivez les tests correspondants.

4 Tests et refactoring

On passe généralement plus de temps à lire/adapter/déterminer du code existant qu'à l'écrire. Cela implique que le temps passé à simplifier son code – le rendre plus lisible – se traduit en gain de temps lors d'une modification ultérieure.

Refactoring

Le **refactoring** (on dit aussi *réusinage de code* ou *remaniement*) est l'opération consistant à retravailler le code source d'un programme informatique – sans toutefois y ajouter des fonctionnalités ni en corriger les bogues – de façon à en améliorer la lisibilité et/ou la modularité afin de faciliter sa maintenance et son évolution.

Pourtant, il est dangereux de modifier un code qui fonctionne. Ne va-t-on pas introduire des erreurs ? Les tests unitaires vont justement nous permettre d'aborder cette opération en toute sérénité ; on apporte une légère amélioration au programme, on relance tous les tests et, si tout passe toujours, on peut poursuivre l'amélioration de notre code. Dès qu'une erreur apparaît, c'est qu'on vient juste de la commettre et on pourra facilement la corriger.

5. En laissant tomber la blessure aléatoire et l'affichage qui est plus difficile à tester.

Exercice 7**Améliorer la lisibilité**

Imaginons que la classe `Dog` contienne une méthode pour vérifier si le poids du chien est compris dans un intervalle donné. En voici une implémentation qui fonctionne mais qui n'est pas très lisible.

```
1  /**
2   * Check if weight is between nb1 and nb2.
3   * If nb1 > nb2, it checks is weight is between nb2 and nb1.
4   *
5   * @param nb1 defines one end of the interval.
6   * @param nb2 defines the other end of the interval.
7   * @return true if dog's weight is between min(nb1,nb2) and max(nb2,nb1) (both included).
8   */
9  boolean isWeightBetween(int nb1, int nb2) {
10     if (nb1 <= nb2) {
11
12     } else {
13         int t = nb1;
14         nb1 = nb2;
15         nb2 = t;
16     }
17     if (!(this.weight < nb1 || nb2 < this.weight)) {
18         return true;
19     } else {
20         return false;
21     }
22 }
```

Et voici les méthodes de tests prévues pour cette méthode.

```
1  @Test
2  void isWeightBetween_nb1_before_nb2_and_weight_is_inside() {
3      Dog dog = new Dog("chien", 10);
4      assertTrue(dog.isWeightBetween(4,24));
5  }
6
7  @Test
8  void isWeightBetween_nb1_before_nb2_and_weight_is_below() {
9      Dog dog = new Dog("chien", 12);
10     assertFalse(dog.isWeightBetween(13,24));
11 }
12
13 @Test
14 void isWeightBetween_nb1_before_nb2_and_weight_is_above() {
15     Dog dog = new Dog("chien", 25);
16     assertFalse(dog.isWeightBetween(13,24));
17 }
18
19 @Test
20 void isWeightBetween_nb1_before_nb2_and_weight_is_nb1() {
21     Dog dog = new Dog("chien", 13);
22     assertTrue(dog.isWeightBetween(13,24));
23 }
24
25 @Test
26 void isWeightBetween_nb1_before_nb2_and_weight_is_nb2() {
27     Dog dog = new Dog("chien", 24);
28     assertTrue(dog.isWeightBetween(13,24));
29 }
30
31 @Test
32 void isWeightBetween_nb1_after_nb2_and_weight_is_inside() {
33     Dog dog = new Dog("chien", 30);
34     assertTrue(dog.isWeightBetween(42,24));
35 }
36
37 @Test
38 void isWeightBetween_nb1_after_nb2_and_weight_is_below() {
39     Dog dog = new Dog("chien", 23);
40     assertFalse(dog.isWeightBetween(33,24));
41 }
```

```

1  @Test
2  void isWeightBetween_nb1_after_nb2_and_weigth_is_above() {
3      Dog dog = new Dog("chien", 34);
4      assertFalse(dog.isWeightBetween(33,24));
5  }
6
7  @Test
8  void isWeightBetween_nb1_after_nb2_and_weigth_is_nb1() {
9      Dog dog = new Dog("chien", 33);
10     assertTrue(dog.isWeightBetween(33,24));
11 }
12
13 @Test
14 void isWeightBetween_nb1_after_nb2_and_weigth_is_nb2() {
15     Dog dog = new Dog("chien", 24);
16     assertTrue(dog.isWeightBetween(33,24));
17 }

```

On vous demande d'apporter les modifications de lisibilité suivantes, une à une, en exécutant les tests après chaque modification afin de vérifier que vous n'introduisez pas de bug.

1. Le premier `if` est vide. Inversez la condition pour ne plus avoir de `else`.
2. Le second test fait intervenir une négation ce qui, pour beaucoup de personnes, le rend plus difficile à comprendre. Réécrivez ce test afin d'éliminer la négation.
3. Une structure de type

```
if (condition) { return true; } else { return false; }
```

peut toujours être simplifiée en un seul `return`. Faites cette amélioration.

5 Développement dirigé par les tests

Développer un code conséquent est une tâche difficile. Nombreux sont les projets informatiques qui n'ont jamais abouti malgré les millions d'Euros et les milliers d'heures investies. Pour se donner les meilleures chances d'aboutir, il faut suivre une **méthodologie éprouvée** et vous aurez des cours de gestion de projets qui vous apprendront les bonnes pratiques.

Un aspect des méthodologies *Agile* – qui sont fort utilisées pour le moment – est ce qu'on appelle le TDD (*Test Driven Development*). Avec cette approche, on écrit d'abord le test pour une nouvelle fonctionnalité **avant** le code qui l'implémente. Alors bien sûr, au début, le test échoue. Le but du jeu est de le faire réussir en apportant la modification nécessaire dans le code. Une fois que c'est fait, on passe à la tâche suivante.

Tutoriel 3

Les années bissextiles

Illustrons cette approche en développant une méthode pour tester si une année est bissextile.

« Une année bissextile [...] est une année comportant 366 jours au lieu des 365 pour une année commune. [...] Les années sont en général bissextiles si elles sont multiples de quatre, toutefois elles ne le sont pas si elles sont multiples de cent à l'exception des années multiples de quatre cents qui sont elles bissextiles. C'est ainsi que les années 2020, 2024 et 2028 sont bissextiles, et que 2000 et 2400 le sont aussi, mais pas 1900, 2100, 2200 et 2300. » Wikipedia⁶.

Nous allons créer une classe de base pour représenter une date du calendrier et y placer notre méthode.

 Créez une nouvelle classe `MyDate`

6. https://fr.wikipedia.org/wiki/Année_bissextile

- ✍ Ajoutez les 3 attributs entiers : `day`, `month` et `year`.
- ✍ Demandez à l'IDE de générer le constructeur.
- ✍ Ajoutez la méthode

```
public boolean isLenient() {
    return false;
}
```

Il s'agit d'un code minimal qui va permettre de compiler mais rien de concret n'est encore écrit dans la méthode.

La plupart des années ne sont pas bissextiles. Faisons en sorte que notre méthode désigne correctement la plupart des années comme non bissextiles.

- ✍ Créez une classe de test.
- ✍ Ajoutez une méthode qui teste une année non bissextile.

```
1  @Test
2  void isLenient_false_when_not_multiple_of_4() {
3      MyDate date = new MyDate(1,1,2003);
4      assertFalse( date.isLenient() );
5  }
```

- ✍ Lancez le test.

Il passe. Évidemment quand on répond n'importe quoi, parfois on tombe juste ;)

Les multiples de 4. Occupons-nous à présent de cette partie de la définition : “*Les années sont en général bissextiles si elles sont multiples de quatre*”. Pour ça :

- ✍ Ajoutez le test suivant :

```
1  @Test
2  void isLenient_true_when_multiple_of_4_but_not_100() {
3      MyDate date = new MyDate(1,1,2004);
4      assertTrue( date.isLenient() );
5  }
```

- ✍ Relancez les tests. Le nouveau test ne passe pas ce qui est normal.
- ✍ Modifiez la méthode afin de tenir compte de la nouvelle contrainte.

```
return year%4==0
```

- ✍ Relancez les tests.

On constate que ces 2 premiers tests passent bien. Il faut maintenant continuer le processus ; on vous le laisse comme exercice ci-dessous...

Exercice 8

Année bissextile

Améliorez le code la méthode `isLenient` du tutoriel précédent en passant par 2 étapes :

1. Prise en compte de “*toutefois elles ne le sont pas si elles sont multiples de cent*”. Ajoutez un test pertinent pour cette contrainte puis modifiez la méthode pour que tous les tests actuels passent.
2. Prise en compte de “*à l'exception des années multiples de quatre cents qui sont elles bissextiles*”. Comme pour le point précédent, ajoutez un test pertinent puis améliorez votre code pour que tous les tests passent.